

Space Mission Risk Analytics

A Calibrated Probability Engine for Launch Success Prediction

Team BetaData

Dhairya Dave | Rudra Bhatt

MSoC 2026 Hackathon — *Data Science*

Microsoft Student Technical Club, DAU

July 8, 2026

Overview

- 1 The Challenge
- 2 Our Solution
- 3 Methodology

- 4 Technology Stack
- 5 Results
- 6 Roadmap

The Challenge

Problem Statement — As understood by Team BetaData

Space launches are among the most capital-intensive activities in engineering. A single failure can erase years of satellite work and hundreds of millions of dollars — yet with 250+ launches a year, mission risk can no longer be judged by manual review alone.

The stakes

- ▶ Decades of mission logs exist, but remain **data-rich, insight-poor**
- ▶ Standard assurance asks only *“Did it succeed?”*
- ▶ No systematic, quantified pre-launch confidence signal exists for planners

The real gap

The question that actually matters:

“How confident should we be, before the countdown starts?”

Objective

- ▶ Turn decades of unstructured launch history into a **calibrated, probabilistic** Go/No-Go signal
- ▶ Ensure the model's confidence scores are **honest** — a “70% success” prediction should succeed roughly 70% of the time
- ▶ Make every prediction **explainable**: not just a risk score, but a reason
- ▶ Give mission planners a live, interactive way to test “What-If” scenarios

In one line

Convert historical launch frequency into predictive, well-calibrated reliability — then explain *why* a mission is flagged, not just *that* it is.

Our Solution

Proposed Solution Overview — A two-layer risk system

1. Predictive Engine

- ▶ Random Forest classifier over vehicle, site, payload, and weather features
- ▶ Isotonic calibration converts raw scores into **true success probabilities**
- ▶ F1-optimal decision threshold, tuned specifically on the failure class

2. Explainability & Simulation

- ▶ Per-mission SHAP attribution in plain language
- ▶ Streamlit dashboard sliced by vehicle / site / decade / payload
- ▶ Interactive “What-If” simulator for mission planners

Design principle

Calibration and honesty over inflated accuracy. A model that is 90% “accurate” by always predicting success is useless — we optimise for **calibrated probabilities** and **honest failure recall** instead.

Methodology

Methodology — Leak-free reliability logic

The golden rule — Temporal Partitioning

Every feature for a launch at time T is computed **strictly** from data available before T . No feature is allowed to peek into the future.

► Vehicle maturity

Days since a rocket family's first launch — young vehicles fail more; reliability grows with flight experience.

► Expanding-window reliability

Rolling success rate per rocket family / site / country, via `.shift(1).expanding().mean()` — unit-tested against leakage.

► Weather enrichment

Real historical weather (wind, temperature, precipitation) from the Open-Meteo API for high-volume sites; unavailable cases flagged, never imputed.

► Era-stratified split

Chronological train/test split — the most recent ~20% of launches held out, mirroring real deployment.

Methodology — From score to honest probability

1. **High-cardinality grouping** — 339 raw rocket families condensed to 92 groups, retaining 80.5% of category-level signal without overfitting a $\sim 4,200$ -row dataset.
2. **Model selection** — Logistic Regression baseline $\Rightarrow$ Random Forest chosen over XGBoost, after XGBoost's calibration step repeatedly collapsed discrimination on this small, imbalanced dataset.
3. **Calibration with a check** — Isotonic regression via `CalibratedClassifierCV`; calibrated vs. uncalibrated ROC-AUC compared at every step.
4. **Threshold, not 0.5** — on a $\sim 90\%$ -success dataset, we pick the F1-optimal cutoff on the *failure* class from the full precision-recall curve.

Model Evolution — Baseline vs. final — real numbers, not rounded guesses

Model	ROC-AUC	Brier	Notes
Logistic Regression	0.72 – 0.73	0.157	class_weight="balanced"
XGBoost (uncalibrated)	0.7364	0.1581	Regularization tuned
XGBoost + Isotonic Calib.	0.5364	0.0547	Rejected — discrimination collapsed
Random Forest + Isotonic	0.7489	0.0472	Final selected model

Why this matters

XGBoost's raw discrimination was fine (0.7364) — it was specifically CalibratedClassifierCV's internal refitting that destroyed it on this small, imbalanced dataset (ROC-AUC fell to 0.5364, barely above chance). Random Forest kept both: strong ranking *and* trustworthy calibrated probabilities. That diagnosis — not just the final number — is the analytical decision this round is meant to test.

Technology Stack

Tools & Technologies

Layer	Tools	Role
Data	Pandas, NumPy	Cleaning raw mission logs; structuring rocket, site, payload fields
Enrichment	Open-Meteo API	Real historical wind / temperature / precipitation per launch site
Modeling	scikit-learn Random Forest	Non-linear interactions across vehicle $\times$ site $\times$ weather
Calibration	CalibratedClassifier (Isotonic)	Converting raw scores into true, verifiable probabilities
Explainability	SHAP (Permutation)	Human-readable, per-mission audit trail
Dashboard	Streamlit	Historical slicing + live What-If simulator
Dataset	Kaggle: <i>All Space Missions from 1957</i>	4,324 launches, 1957–2020

Results

Results Snapshot — Calibration & honest signal — held-out test set, 840 launches

ROC-AUC 0.75

identical calibrated vs. uncalibrated

Brier 0.0472

down from 0.1648 uncalibrated

PR-AUC 0.3306

failure class — $\sim 5\text{--}6\times$ random

Mean Calibration Error: **0.037** — predicted probabilities differ from observed frequencies by only ~ 3.7 percentage points on average.

Why not higher — and why that's the honest answer

Rocket failures are driven substantially by engineering and manufacturing factors that no public historical metadata captures. This model estimates **systemic risk** (vehicle maturity, site experience, era) rather than the specific fault of any one flight. A PR-AUC of ~ 0.33 using only metadata is genuine signal, not an inflated accuracy claim.

Operating Threshold — F1-optimal on the failure class, not the default 0.5

Class	Precision	Recall	F1-score
Failure (n = 48)	0.48	0.33	0.40
Success (n = 792)	0.96	0.98	0.97

- ▶ Chosen threshold: failure probability $\geq$ **0.241** (test set: 840 launches, 48 failures, 5.7% base rate)
- ▶ When flagged high-risk, the model is correct **48% of the time** — roughly **8.4×** the baseline 5.7% failure rate
- ▶ It surfaces about **one in three** historical failures — a practical screening signal, not a certainty claim
- ▶ Threshold is versioned in `models/model_metadata.json` (`chosen_threshold_failure_prob`) and consumed by `src/predict.py`

Documented, Not Hidden — Data limitations

Limitation	Our mitigation
No payload mass column	Public-knowledge lookup table, flagged when using fallback values
Weather covers only 18.1% of launches	Deliberate scope to high-volume sites; rest flagged <code>weather_available=False</code> , never imputed
4-class status collapsed to binary	Documented simplification; sample sizes too small for finer classes
~4,200 launches, ~230 failures	Conservative regularization + honest PR-AUC vs. base-rate reporting

Roadmap

Tentative Roadmap — Round 2

